



รายงานโครงงานการออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack
กรณีศึกษา กลุ่มที่ 6 Warehouse Retrieval: Multi-Stack and Greedy
Selection

ผู้จัดทำ

1. กษิตศ ชุมแวงวาปี 66122250103
2. ศักกรินทร์ น้อยอุดม 66122250104
3. นัทธพงศ์ ราชวงศ์ 67122250021
4. อธิภัทร ทองจิตร 68122250017
5. ภูวนนท์ บริดมอร์ 68122250056
6. ดลเดชธนพัฒน์ จิตะฐาน 68122250062
7. วรกร ชัยยะอธิปัตย์ 68122250077

เสนอ

อาจารย์กัลยณัฐ คุหลาบเพชรทอง

รายงานนี้เป็นส่วนหนึ่งของวิชา CSD2103 การออกแบบและวิเคราะห์ขั้นตอนวิธี

สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยีมหาวิทยาลัยราชภัฏสวนสุนันทา

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

การบริหารจัดการพื้นที่ในคลังสินค้า (Warehouse) ถือเป็นปัจจัยสำคัญในการลดต้นทุนและเพิ่มประสิทธิภาพในการทำงาน เมื่อพื้นที่จัดเก็บมีจำกัด การวางสินค้าซ้อนกันเป็นกอง (Stack) จึงเป็นวิธีที่ได้รับความนิยม อย่างไรก็ตาม ปัญหาความล่าช้าจะเกิดขึ้นเมื่อต้องการนำกล่องสินค้าเป้าหมายที่อยู่ด้านล่างสุดออกมา เนื่องจากต้องทำการย้ายกล่องที่วางทับอยู่ด้านบนไปยังกองอื่นชั่วคราว โครงงานนี้จึงจัดทำขึ้นเพื่อประยุกต์ใช้โครงสร้างข้อมูลแบบ Stack ในการจำลองและวิเคราะห์อัลกอริทึมการดึงกล่องสินค้าเป้าหมายเพื่อเปรียบเทียบประสิทธิภาพในการย้ายกล่องระหว่างอัลกอริทึมสองรูปแบบ

1.2 Case Story

ระบบคลังสินค้ามีการวางกล่องซ้อนกันเป็นหลายกอง โดยกล่องที่อยู่ด้านบนสามารถนำออกได้ทันที แต่หากกล่องเป้าหมายอยู่ด้านล่าง ต้องย้ายกล่องที่ขวางอยู่ไปยัง Stack อื่นก่อน และหลังจากนำกล่องเป้าหมายออกแล้ว ต้องสามารถนำกล่องที่ย้ายกลับคืนมาได้ ปัญหาหลักคือการออกแบบอัลกอริทึมเพื่อนำกล่องเป้าหมายออก โดยเปรียบเทียบวิธีเลือก Stack ปลายทาง 2 วิธี ได้แก่ Algorithm A: First Available Stack (เลือก Stack แรกที่มีพื้นที่ว่าง) และ Algorithm B: Best Fit Stack (เลือก Stack ที่เหลือพื้นที่ว่างน้อยที่สุดหลังจากวางกล่อง)

1.3 วัตถุประสงค์

1. เพื่อวิเคราะห์ปัญหาและออกแบบอัลกอริทึมสำหรับย้ายกล่องในคลังสินค้าด้วยโครงสร้างข้อมูล Stack
2. เพื่อพัฒนาและเปรียบเทียบอัลกอริทึม First Available Stack และ Best Fit Stack
3. เพื่อวิเคราะห์ Time Complexity และ Space Complexity ของทั้งสองอัลกอริทึม
4. เพื่อพัฒนาโปรแกรมภาษา Java และทดลองประสิทธิภาพการทำงานจริง

1.4 ขอบเขตงาน

1. ย้ายได้เฉพาะกล่องด้านบนสุดของ Stack เท่านั้น
2. แต่ละ Stack มีความจุจำกัด
3. Stack ปลายทางต้องมีพื้นที่ว่างเพียงพอให้วางกล่องได้
4. หลังนำกล่องเป้าหมายออก ต้องสามารถนำกล่องที่ย้ายกลับคืนได้
5. ไม่บังคับใช้เงื่อนไขน้ำหนักกล่องในการคำนวณ

2.1 ข้อมูลนำเข้า (Input)

ข้อมูลนำเข้าของระบบประกอบด้วย:

ข้อมูลคลังสินค้า : จำนวน Stack ทั้งหมด (k), จำนวนกล่องทั้งหมดในระบบ (n), และความจุสูงสุดของแต่ละ Stack

ข้อมูลกล่อง (Box Data): ประกอบด้วย Box ID, Category, Arrival Time และ Priority

คำสั่งเป้าหมาย: รหัสกล่องเป้าหมาย (Target Box) ที่ต้องการนำออกจากคลังสินค้า

2.2 ผลลัพธ์ที่ต้องการ (Output)

นำกล่องเป้าหมายออกจาก Stack ได้สำเร็จ

กล่องอื่นๆ ที่ถูกย้ายเพื่อเปิดทาง ถูกนำกลับมาจัดเก็บอย่างถูกต้อง

2.3 ข้อจำกัดของปัญหา (Constraints)

การเข้าถึงข้อมูลต้องเป็นไปตามคุณสมบัติของ Stack (LIFO - Last In, First Out) กล่าวคือเข้าถึงได้เฉพาะกล่องบนสุด

ไม่สามารถย้ายกล่องไปยัง Stack ที่มีพื้นที่เต็มแล้วได้

2.4 ข้อตกลงที่ใช้ (Assumptions)

ในกรณีที่จำเป็นต้องย้ายกล่อง ระบบมีหน่วยความจำหรือตัวแปรชั่วคราวเพื่อจดจำลำดับของกล่องและ Stack ต้นทาง เพื่อให้สามารถย้ายกลับได้อย่างถูกต้องตามลำดับเดิม

2.5 กรณีขอบเขต (Edge Cases) และการจัดการข้อผิดพลาด

ระบบจำเป็นต้องรองรับและจัดการกรณีต่างๆ ดังนี้:

กรณีปกติ: กล่องเป้าหมายอยู่ตรงกลางของ Stack

กรณีขอบเขต (Edge Cases): กล่องเป้าหมายอยู่ด้านบนสุด หรือกล่องเป้าหมายอยู่ด้านล่างสุดของ Stack

กรณีที่อาจทำให้โปรแกรมผิดพลาด: ไม่พบกล่องเป้าหมายในระบบ, Stack ปลายทางบางกองเต็ม, ไม่มีพื้นที่เพียงพอสำหรับย้ายกล่อง หรือมี Stack ว่าง

2.6 การวิเคราะห์ปฏิบัติการ (Operations Analysis)

Operation ที่เกิดขึ้นบ่อย: ปฏิบัติการ Pop (นำกล่องออก) และ Push (ใส่กล่องเข้า) เพื่อย้ายกล่องที่ขวางอยู่

Operation ที่อาจใช้เวลามากที่สุด: ปฏิบัติการค้นหาและตรวจสอบพื้นที่ว่างของ Stack ปลายทาง (โดยเฉพาะวิธี Best Fit Stack ที่ต้องตรวจสอบพื้นที่ว่างของทุก Stack เพื่อเปรียบเทียบ) และการย้ายกล่องที่อยู่เหนือเป้าหมายจำนวน h ใบ รวมถึงการย้ายกล่องกลับคืนเมื่อนำกล่องเป้าหมายออกไปแล้ว

3.1 ทบทวนขอบเขตปัญหา

จากกรณีศึกษาคลังสินค้าจัดเก็บกล่องเป็นหลายกอง(Stack) โดยกล่องที่อยู่บนสุดของแต่ละกองสามารถหยิบออกได้ทันทีตามคุณสมบัติ LIFO (Last In, First Out) ของ Stack แต่หากกล่องเป้าหมาย (Target Box) ถูกฝังอยู่ใต้กล่องอื่น ระบบจำเป็นต้องย้ายกล่องที่ขวางอยู่ด้านบนไปพักไว้ยัง Stack อื่นก่อน จึงจะสามารถดึงกล่องเป้าหมายออกมาได้และเมื่อถึงเสร็จแล้วต้องนำกล่องที่ย้ายออกไปกลับมาจัดเรียงในลำดับเดิม ตารางที่

3.1 สรุปข้อมูลนำเข้า ผลลัพธ์ที่ต้องการ และข้อจำกัดของปัญหาโดยย่อ ก่อนเข้าสู่รายละเอียดการออกแบบอัลกอริทึมทั้งสองวิธี

หัวข้อ	รายละเอียด
Input	จำนวน Stack (k), จำนวนกล่องทั้งหมด (n), ความจุสูงสุดต่อ Stack, ข้อมูลกล่อง (Box ID, Category, Arrival Time, Priority) และรหัสกล่องเป้าหมาย
Output	กล่องเป้าหมายถูกนำออกสำเร็จ และกล่องอื่นที่ถูกย้ายเพื่อเปิดทางถูกนำกลับไปจัดเก็บในลำดับเดิม
Constraints	เข้าถึงได้เฉพาะกล่องบนสุดของแต่ละ Stack (LIFO) และห้ามย้ายกล่องไปยัง Stack ที่มีพื้นที่เต็มแล้ว
Assumptions	ระบบมีหน่วยความจำ/โครงสร้างข้อมูลชั่วคราวสำหรับจดจำลำดับกล่องและ Stack ต้นทาง เพื่อให้ย้ายกลับได้ถูกต้องตามลำดับเดิม
Edge Cases	กล่องเป้าหมายอยู่บนสุด/ล่างสุด, ไม่พบกล่องเป้าหมาย, Stack ปลายทางเต็มทุกกอง, พื้นที่รวมไม่พอสำหรับย้ายกล่องขวาง

ด้วยข้อจำกัดข้างต้น ทีมผู้พัฒนาจึงออกแบบอัลกอริทึมสองวิธีที่แก้ปัญหเดียวกันแต่ใช้กลยุทธ์ต่างกัน ได้แก่ Algorithm A ซึ่งค้นหา Stack ปลายทางด้วยกติกา First Available และ Algorithm B ซึ่งใช้กติกา Best Fit ร่วมกับการเรียกซ้ำ (Recursion) โดยทั้งสองวิธีมีเป้าหมายผลลัพธ์เดียวกันแต่ต่างกันที่โครงสร้างการค้นหาและกติกาการเลือก Stack ปลายทาง

3.2 Algorithm A — Multi-Stack Retrieval ด้วยวิธี First Available (Iterative)

3.2.1 แนวคิดหลัก

Algorithm A ใช้วิธีการแบบวนซ้ำ (Iterative) ร่วมกับโครงสร้างข้อมูลสำรองแบบ ExplicitStack(tempMovedList) เพื่อพักกล่องที่ถูกย้ายออกชั่วคราว เมื่อพบว่ามิกล่องวางอยู่เหนือกล่องเป้าหมายอัลกอริทึมจะไล่ค้นหา Stack เริ่มต้นตั้งแต่กองแรกไปจนถึงกองสุดท้าย (index 0 ถึง k-1 โดยข้าม Source Stack) และเลือก Stack แรกที่ยังมีพื้นที่ว่างทันที (First Available) จากนั้นย้ายกล่องไปเก็บไว้ที่ Stack นั้นหากไม่พบ Stack ที่มีพื้นที่ว่างเพียงพอระบบจะทำRollbackคือนำกล่องที่ย้าย

ไปแล้วทั้งหมดกลับคืนSourceStackตามลำดับเดิม แล้วยกเลิกการดำเนินการ

3.2.2 Pseudocode

```
ALGORITHM RetrieveBox_FirstAvailable(stacks, target)
```

```
INPUT: stacks      : รายการ Stack ทั้งหมดในคลังสินค้า
```

target : รหัสกล่องเป้าหมาย (Box ID)

OUTPUT: boolean : true หากดึงกล่องเป้าหมายออกสำเร็จ, false หากไม่สำเร็จ

```
1. sourceStack ← FindStackContaining(stacks, target)
2. IF sourceStack = null THEN RETURN false
3. tempMovedList ← empty list
4. WHILE sourceStack.peek().id ≠ target DO
5.     box ← sourceStack.pop()
6.     destStack ← null
7.     FOR EACH s IN stacks WHERE s ≠ sourceStack DO
8.         IF s.hasSpace() THEN
9.             destStack ← s
10.            BREAK // First Available: หยุดค้นหาทันทีที่พบ
11.        END IF
12.    END FOR
13.    IF destStack = null THEN
14.        Rollback(tempMovedList, sourceStack)
15.        RETURN false
16.    END IF
17.    destStack.push(box)
18.    tempMovedList.push( (box, destStack) )
19. END WHILE
20. sourceStack.pop() // ดึงกล่องเป้าหมายออกสำเร็จ
21. Restore(tempMovedList, sourceStack) // คืนกล่องกลับตามลำดับย้อนกลับ (LIFO)
```

22. RETURN true

3.2.3 กฎการเลือก Stack ปลายทาง — First Available Selection

ตรวจสอบ Stack ตั้งแต่ดัชนี 0 ถึง $k-1$ ตามลำดับ

ข้าม Source Stack (ไม่นำมาพิจารณาเป็นปลายทาง)

ตรวจสอบว่า Stack ที่พิจารณามีพื้นที่ว่างหรือไม่

หากมีพื้นที่ว่าง ให้เลือก Stack นั้นทันทีและ BREAK ออกจากการค้นหา

หากไม่มี Stack ใดมีพื้นที่ว่างเลย ให้ทำ Rollback และคืนค่า false

ตัวอย่างเช่น เมื่อ Source Stack คือ Stack 0 อัลกอริทึมจะข้าม Stack 0 แล้วตรวจสอบ Stack 1 หากมีพื้นที่ว่างจะเลือกทันทีโดยไม่ตรวจสอบ Stack ถัดไปอีก

3.2.4 ตัวอย่าง Step-by-step Trace

ขั้นตอน	สถานะ Stack 0 (Source)	สถานะ Stack 1	สถานะ Stack 2
Initial	$C \rightarrow B \rightarrow A$ (Target)	ว่าง	ว่าง
Move C	$B \rightarrow A$ (Target)	C	ว่าง
Move B	A (Target)	$B \rightarrow C$	ว่าง
Retrieve A	ว่าง (Pop A สำเร็จ)	$B \rightarrow C$	ว่าง
Restore	$C \rightarrow B$ (คืนตามลำดับ LIFO)	ว่าง	ว่าง

ตาราง 3.1 ตัวอย่างการทำงานของ Algorithm A: กล่อง A ถูกดึงออกสำเร็จ และกล่อง C, B กลับสู่ลำดับเดิมบน Stack 0

3.2.5 Restore และ Rollback

Restore: เมื่อสามารถนำกล่องเป้าหมายออกได้สำเร็จ ระบบจะนำกล่องที่ถูกย้ายไปยัง Stack อื่นกลับมายัง Source Stack โดยอ่านข้อมูลจาก tempMovedList แล้วนำกลับในลำดับย้อนกลับ (LIFO) เพื่อรักษาลำดับเดิมของกล่องให้ถูกต้อง 100%

Rollback: หากไม่สามารถหา Stack ปลายทางที่มีพื้นที่ว่างได้ ระบบจะนำกล่องที่ย้ายไปแล้วทั้งหมดกลับคืน Source Stack ก่อน แล้วจึงคืนค่า false เพื่อให้สถานะคลังสินค้าไม่เปลี่ยนแปลงจากก่อนเริ่มการทำงาน

3.3 Algorithm B — Multi-Stack Retrieval ร่วมกับ Greedy Selection ด้วยวิธี Best Fit (Recursive)

3.3.1 แนวคิดหลัก

Algorithm B ใช้การเรียกซ้ำ (Recursion) แทนการวนลูปแบบ Explicit เพื่อไล่ค้นหากล่องเป้าหมายลึกลงไปใน Stack โดยอาศัย Call Stack ของระบบ (ImplicitStack) เป็นพื้นที่จดจำลำดับกล่องที่ถูกดึงออกชั่วคราวในแต่ละขั้นตอนของการเรียกฟังก์ชัน จุดที่ต่างจาก Algorithm A อย่างชัดเจนคือกติกาการเลือก Stack ปลายทาง: แทนที่จะเลือก Stack แรกที่พบว่ามีพื้นที่ว่าง (First Available) Algorithm B จะสำรวจ Stack ที่เป็นไปได้ทั้งหมดแล้วเลือก Stack ที่เหลือพื้นที่ว่างน้อยที่สุดแต่ยังพอสำหรับกล่องที่จะย้าย (Best Fit) ซึ่งเป็นแนวคิดแบบ Greedy ที่มุ่งใช้พื้นที่ให้คุ้มค่าที่สุดในแต่ละก้าวและเก็บ Stack ที่มีพื้นที่ว่างมากไว้สำหรับการย้ายกล่องล็อตใหญ่ในอนาคต

แม้ Algorithm B จะไม่ต้องประกาศโครงสร้าง Stack สำรองสำหรับ "ลำดับการดึงกล่อง" เนื่องจากใช้ CallStack ของระบบแทน แต่เนื่องจากโจทย์กำหนดให้กล่องต้องถูกย้ายไปเก็บจริงใน Stack ภายนอกอื่น (ไม่ใช่แค่พักไว้ในหน่วยความจำ) จึงยังคงต้องมีการ

บันทึกคู่ (กล่อง, Stack ปลายทาง) ไว้ในmovedList เพื่อใช้ตอนRestoreเช่นเดียวกับ Algorithm A ความแตกต่างที่แท้จริงระหว่างสองวิธีจึงอยู่ที่ (1) โครงสร้างการค้นหา — Loop กับ Explicit Stack เทียบกับ Recursion บน Call Stack และ (2) กติกาการเลือก Stack ปลายทาง — First Available เทียบกับ Best Fit (Greedy Selection)

3.3.2 Pseudocode B

```
ALGORITHM RetrieveBox_BestFitRecursive(sourceStack, stacks, target,
movedList)
```

INPUT: sourceStack : Stackที่กำลังค้นหากล่องเป้าหมายอยู่ในขณะนี้

stacks : รายการ Stack ทั้งหมดในคลังสินค้า

target : รหัสกล่องเป้าหมาย (Box ID)

movedList : รายการบันทึก (กล่อง, Stack ปลายทาง) ที่ถูกย้าย — ใช้ร่วมกันทุกชั้นการเรียกซ้ำ

OUTPUT: boolean : true หากพบและดึงกล่องเป้าหมายออกสำเร็จ, false หากไม่พบ

```
1. IF sourceStack.isEmpty() THEN RETURN false
```

```
2. IF sourceStack.peek().id = target THEN
```

```
3. sourceStack.pop() // ดึงกล่องเป้าหมายออกสำเร็จ ณ
```

```
4. RETURN true
```

5. END IF

```
6. box ← sourceStack.pop()           // ยกกล่องวางออกชั่วคราว
(พักใน Call Stack Frame)
```

```
7. destStack ← FindBestFitStack(stacks, sourceStack, box) // Greedy:
   เหลือพื้นที่น้อยที่สุดแต่พอ
```

```

8. IF destStack = null THEN
9.   sourceStack.push(box)           // คีนกล่องท้นที่ ยังไม่มีที่ให้ย้าย
10.  RETURN false
11. END IF
12. destStack.push(box)
13. movedList.push( (box, destStack) )
14. found ← RetrieveBox_BestFitRecursive(sourceStack, stacks, target,
movedList) // เรียกตัวเองซ้ำ
15. IF found = false THEN
16.   Rollback(movedList, sourceStack) // คีนกล่องทั้งหมดกลับ
ยกเลิกการทำงาน
17. RETURN found

// เมื่อ Top-level call คีนค่า true ผู้เรียกจะสั่ง Restore(movedList,
sourceStack)
// เพื่อคีนกล่องที่ย้ายไว้กลับมาตามลำดับย้อนกลับ (LIFO) เช่นเดียวกับ Algorithm
A

```

3.3.3 กฎการเลือก Stack ปลายทาง — Best Fit Selection (Greedy)

สำรวจ Stack ทั้งหมดที่ไม่ใช่ Source Stack และยังมีพื้นที่ว่างเหลืออยู่

คำนวณพื้นที่ว่างคงเหลือของแต่ละ Stack หลังสมมติว่าย้ายกล่องเข้าไปแล้ว

เลือก Stack ที่มีพื้นที่ว่างคงเหลือน้อยที่สุด (Tightest Fit) แต่ยังไม่ล้น —
เป็นการตัดสินใจแบบ Greedy ในแต่ละก้าว

หากไม่มี Stack ไดรองรับได้เลย ให้คีนค่า null เพื่อให้ผู้เรียกทำ Rollback

กติกานี้ต่างจากAlgorithmAตรงที่ต้องตรวจสอบStack ที่เหลือทั้งหมดก่อนจึงตัดสินใจได้(ไม่BREAKทันทีที่พบStackว่าง) ซึ่งส่งผลต่อเวลาในการทำงานดังจะกล่าวถึงในบทที่ 5

3.3.4 การทำงานของ Algorithm B ตามลำดับ Call Stack

1.ตรวจสอบเงื่อนไขจุดหยุด(BaseCaseCheck):หากStack ว่างแสดงว่าไม่พบกล่องเป้าหมายและคืนค่าfalseแต่ถ้ากล่องบนสุดตรงกับเป้าหมาย จะดึงออกและคืนค่า trueทันที

2.ดึงกล่องบนสุดออกชั่วคราว(PopItem):หากยังไม่พบเป้าหมาย โปรแกรมจะดึงกล่องบนสุดออกและหาStackปลายทางแบบBest Fit เพื่อย้ายไปเก็บจริง

3.เรียกตัวเองซ้ำเพื่อค้นหาลึกลงไป(RecursiveCall): ฟังก์ชันเรียกตัวเองซ้ำเพื่อตรวจสอบกล่องลำดับถัดไปที่อยู่ลึกกว่าใน Source Stack

4.ย้อนกลับและคืนกล่อง(Backtracking/Restore): เมื่อการค้นหาในชั้นลึกกว่าเสร็จสิ้นและส่งผลลัพธ์กลับขึ้นมาหากพบเป้าหมายสำเร็จ ระบบจะใช้ movedList นำกล่องทั้งหมดที่ย้ายไปกลับคืน Source Stack ตามลำดับเดิม 100%

3.4 เปรียบเทียบแนวคิดหลักระหว่าง Algorithm A และ Algorithm B

หัวข้อเปรียบเทียบ	Algorithm A (Iterative + First Available)	Algorithm B (Recursive + Best Fit)
โครงสร้างการค้นห	Loop(While) ร่วมกับExplicit Stack (tempMovedList)	Recursionโดยใช้CallStack ของระบบ (Implicit Stack)

หัวข้อเปรียบเทียบ	Algorithm A (Iterative + First Available)	Algorithm B (Recursive + Best Fit)
กติกาเลือก Stack ปลายทาง	FirstAvailable เลือกกองแรกที่พบว่า มีที่ว่าง	BestFit สํารวจทุกกองแล้วเลือกกองที่ เหลือพื้นที่น้อยที่สุดแต่พอ
จำนวน Stack ที่ ต้องตรวจต่อการ ย้าย 1 กอง	เฉลี่ยน้อยกว่า (หยุด ทันทีที่พบ)	ต้องตรวจ Stack ที่เหลือทั้งหมดทุกครั้ง
ความเสี่ยง Stack Overflow	ต่ำ (ใช้หน่วยความจำ Heap ผ่าน List)	มีความเสี่ยงหาก n มีขนาดใหญ่มาก เนื่องจากความลึกของ Recursion
ลักษณะโค้ด	ตรงไปตรงมา อ่าน ง่าย แต่ต้องเขียน Loop คำนวณเพิ่มเอง	กระชับ สั้น แต่ต้องเข้าใจกลไก Call Stack
ผลต่อการใช้พื้นที่ คลังสินค้า	อาจเหลือพื้นที่ กระจัดกระจาย เพราะไม่คำนึง ถึงขนาดที่เหลือ	ใช้พื้นที่คุ้มค่ากว่าในระยะยาว เพราะ เลือกแบบเจาะจง

ตาราง 3.2 สรุปเปรียบเทียบแนวคิดหลักของ Algorithm A และ Algorithm B

บทที่ 4 การอธิบายความถูกต้อง (Correctness)

บทนี้นำเสนอการพิสูจน์ความถูกต้องของ Algorithm A และ Algorithm B โดยใช้เทคนิคที่เหมาะสมกับโครงสร้างการทำงานของแต่ละวิธี กล่าวคือ Algorithm A ซึ่งเป็นแบบวนซ้ำใช้เทคนิค Loop Invariant ส่วน Algorithm B ซึ่งเป็นแบบเรียกซ้ำใช้เทคนิค Precondition / Postcondition ร่วมกับการวิเคราะห์ Base Case และ Recursive Case ทั้งสองแนวทางล้วนต้องแสดงองค์ประกอบสี่ส่วนเดียวกันได้แก่ Precondition, Invariant, Postcondition และ Termination

4.1 Precondition

มี Stack ของคลังสินค้าอย่างน้อย 1 กอง และแต่ละกองมีความจุสูงสุดที่ทราบค่า
รหัสกล่องเป้าหมาย (target) เป็นค่าที่ถูกต้องและไม่เป็นค่าว่าง
โครงสร้างข้อมูลของกล่องในทุก Stack สอดคล้องกับคุณสมบัติ LIFO
ก่อนเริ่มการทำงาน

4.2 Invariant

4.2.1 Loop Invariant ของ Algorithm A

Initialization: ก่อนเริ่มลูป ยังไม่มีการย้ายกล่องใด ๆ ดังนั้น tempMovedList จึงว่างเปล่า ซึ่งถือว่าถูกต้องตามเงื่อนไขเริ่มต้น

Maintenance: ในทุกรอบของลูป กล่องที่ถูกดึงออกจาก Source Stack จะถูกบันทึกคู่กับ Stack ปลายทางไว้ใน tempMovedList เสมอ ทำให้เมื่อสิ้นสุดลูป ระบบมี "แผนที่" ครบถ้วนสำหรับนำกล่องทุกใบกลับไปวางที่เดิมในลำดับที่ถูกต้อง (LIFO)

Termination: เมื่อกล่องบนสุดของ Source Stack ตรงกับเป้าหมาย หรือเมื่อไม่พบ Stack ปลายทางที่รองรับได้ ลูบจะหยุดทำงาน และระบบจะดำเนินการ Restore หรือ Rollback ให้ Stack กลับสู่สภาพที่ถูกต้อง 100%

4.2.2 Recursive Invariant ของ Algorithm B (Pre/Post Condition ต่อการเรียกแต่ละชั้น)

Pre-conditionของแต่ละชั้นการเรียก:sourceStack ที่ส่งเข้ามามีลำดับกล่องที่ถูกต้องตามสภาพจริงและmovedListสะสมค่าคู่ (กล่อง, ปลายทาง) จากทุกชั้นก่อนหน้าอย่างครบถ้วน

Recursive step: หากกล่องบนสุดไม่ใช่เป้าหมาย ให้ย้ายกล่องนั้นไปยัง Stack ที่ดีที่สุดตามกติกา Best Fit แล้วเรียกฟังก์ชันเดิมซ้ำเพื่อพิจารณากล่องถัดไป ซึ่งรักษาคุณสมบัติเดิมไว้ในทุกชั้นของการเรียก

Post-condition ของแต่ละชั้นการเรียก: เมื่อฟังก์ชันชั้นนั้นคืนค่า ไม่ว่าจะเป็ trueหรือfalseสถานะของStackทั้งหมดต้องสอดคล้องกับค่าที่คืน กล่าวคือหากคืนค่า false ต้องไม่มีร่องรอยการย้ายที่ยังค้างอยู่จากชั้นนั้น (มีการ Rollback แล้ว)

4.3 Postcondition

หากพบกล่องเป้าหมาย:กล่องเป้าหมายถูกดึงออกจากคลังสินค้าเรียบร้อยแล้ว และกล่องอื่นทุกใบที่เคยถูกย้ายกลับคืนสู่Stack เดิมในลำดับเดิมทุกประการ (สถานะ Stack เทียบเท่าการดึงกล่องเป้าหมายออกจากลำดับเดิมโดยตรง)

หากไม่พบกล่องเป้าหมาย หรือไม่มีพื้นที่เพียงพอสำหรับย้าย: สถานะของทุก Stack ต้องเหมือนกับก่อนเริ่มการทำงานทุกประการ (ไม่มีผลข้างเคียงตกค้าง)

4.4 Termination

ทั้งสองอัลกอริทึมใช้ตัวชี้วัดขนาดปัญหา(measure)เดียวกันคือ "จำนวนกล่องที่อยู่เหนือกล่องเป้าหมายในSourceStack" ซึ่งมีค่าเป็นจำนวนเต็มไม่ติดลบและมีขอบเขตบนคือ n (จำนวนกล่องทั้งหมด)

Algorithm A: ทุกรอบของ WHILE loop จะดึงกล่องออกจาก Source Stack หนึ่งใบเสมอ (บรรทัดที่ 5 ในผังเทียม) ทำให้ค่า measure ลดลงอย่างน้อย 1 หน่วยทุกรอบ เมื่อ measure ลดถึง 0 (พบเป้าหมายที่ตำแหน่งบนสุด) ลูปจะยุติ จึงรับประกันว่าลูปทำงานไม่เกิน n รอบ

AlgorithmB:ทุกการเรียกซ้ำที่ไม่ใช่BaseCaseจะpop กล่องออกหนึ่งใบก่อนเรียกตัวเองซ้ำ (บรรทัดที่ 6 และ 14) ทำให้ความลึกของ Recursion ลดค่าmeasure ลงหนึ่งหน่วยทุกครั้ง และจะพบ Base Case (Stack ว่าง หรือกล่องบนสุดตรงเป้าหมาย)ภายในความลึกไม่เกิน n ชั้น จึงรับประกันการยุติเช่นเดียวกัน

จากการพิสูจน์ทั้งสองประกอบข้างต้น สรุปได้ว่าทั้ง Algorithm A และ Algorithm B มีความถูกต้อง(Correct)ตามนิยามกล่าวคือทำงานเสร็จสิ้นเสมอ(Termination) และให้ผลลัพธ์ตรงตามข้อกำหนด(Postcondition)ภายใต้เงื่อนไขเริ่มต้นที่กำหนด(Precondition) ในทุกกรณี

บทที่ 5 การวิเคราะห์ประสิทธิภาพ (Complexity Analysis)

5.1 Algorithm A: กลยุทธ์ "ฝากของไว้ข้างนอก" (Iterative + Explicit Stack)

Best Case: $O(1)$ — กล่องเป้าหมายอยู่บนสุดพอดี ไม่ต้องย้ายกล่องใด ๆ

Average Case: $O(n/2) \approx O(n)$ — โดยเฉลี่ยกล่องเป้าหมายอยู่กึ่งกลางกอง ต้องย้ายกล่องประมาณครึ่งหนึ่งของจำนวนกล่องใน Stack นั้น

Worst Case: $O(n)$ กล่องเป้าหมายอยู่ล่างสุด หรือไม่พบกล่องเป้าหมายเลย ต้องตรวจ/ย้ายกล่องทุกใบ

Space Complexity: $O(n)$ ต้องใช้พื้นที่ tempMovedList (Explicit Stack) ขนาดสูงสุดเท่ากับจำนวนกล่องที่ถูกย้ายทั้งหมด

5.2 Algorithm B: กลยุทธ์ "จดจำไว้ในใจ" (Recursive + Best Fit)

Best Case: $O(1)$ — กล่องเป้าหมายอยู่บนสุด พบและคืนค่าใน Base Case ทันที

Average Case: $O(n/2)$ ต่อการค้นหา คุณด้วยต้นทุนการสำรวจ Stack ปลายทางแบบ Best Fit ที่ $O(k)$ ต่อครั้ง (k = จำนวน Stack) จึงมีค่าคงที่ (constant factor) สูงกว่า Algorithm A แม้ Big-O ของทั้งคู่จะเท่ากัน

Worst Case: $O(n)$ — ต้องเรียกซ้ำจนถึงกล่องใบสุดท้าย หรือไม่พบกล่องเป้าหมาย

Space Complexity: $O(n)$ — แม้ไม่มีการประกาศ Explicit Stack แต่ระบบใช้พื้นที่ CallStack ตามความลึกของ Recursion ซึ่งแปรผันตรงกับจำนวนกล่องที่ต้องดึงออกก่อนพบเป้าหมาย

5.3 ตารางเปรียบเทียบ Time และ Space Complexity

หัวข้อ	AlgorithmA (Iterative / First Available)	Algorithm B (Recursive / Best Fit)
Best Case (Time)	$O(1)$	$O(1)$
Average Case (Time)	$O(n)$	$O(n)$ — ค่าคงที่สูงกว่าเนื่องจากสำรวจ Stack ปลายทางทุกกอง
Worst Case (Time)	$O(n)$	$O(n)$
Space Complexity	$O(n)$ — Explicit Stack (tempMovedList)	$O(n)$ — Call Stack (วลีของ Recursion)
ต้นทุนต่อการเลือก Stack ปลายทาง	$O(k)$ แต่หยุดทันทีที่พบ (มักน้อยกว่า k)	$O(k)$ ครบทุกครั้ง (ต้องเทียบทุกกอง)
ความเสี่ยงเชิงพื้นที่ระบบ	ต่ำ ใช้ Heap ผ่าน List/ArrayList	สูงกว่าเมื่อ n มาก เสี่ยง StackOverflowError

ตาราง 5.1 สรุปเปรียบเทียบ Time Complexity และ Space Complexity ของ Algorithm A และ B

แม้ Big-O เชิงทฤษฎีของทั้งสองวิธีจะอยู่ในระดับ $O(n)$ เท่ากันทั้ง Time และ Space Complexity แต่ค่าคงที่ (constant factor) ของ Algorithm B สูงกว่าเนื่องจากทุกครั้งที่ต้องย้ายกล่องหนึ่งใบ อัลกอริทึมต้องสำรวจ Stack ที่เหลือทั้งหมดเพื่อหา Best Fit ในขณะที่ Algorithm A เพียงหยุดค้นหาทันทีที่พบ Stack แรกที่มีพื้นที่ว่าง

ความแตกต่างนี้จะปรากฏชัดเจนในผลการทดลองจริงที่รายงานในบทที่ 7

บทที่ 6 การพัฒนาโปรแกรม Java (Implementation)

6.1 Class Diagram

โครงสร้างโปรแกรมแบ่งความรับผิดชอบออกเป็น 6 คลาสหลัก ดังแผนภาพด้านล่าง



```

| - capacity: int          |
+-----+
| + push(box: Box): void   |
| + pop(): Box            |
| + peek(): Box           |
| + isEmpty(): boolean    |
| + hasSpace(): boolean   |
| + remainingSpace(): int  |
| + getBoxes(): Deque<Box> |
+-----+

```

^

```

| 1..k

```

```

|

```

```

+-----+
|      Warehouse      |
+-----+
| - stacks: List<WarehouseStack> |
+-----+
| + addStack(stack): void         |
| + getStacks(): List<...>       |
| + findStackContaining(id)      |
| + validate(): void            |
+-----+

```

^

^



Both Algorithms use:

```

+-----+
|      MovedBox      |
+-----+
| - box: Box          |
| - stack: WarehouseStack |
+-----+
| + box(): Box        |
| + stack(): WarehouseStack |
+-----+

```

ภาพ 6.1 Class Diagram (แสดงในรูปแบบข้อความ) ของระบบ Warehouse Retrieval

6.2 โครงสร้างคลาสหลัก

6.2.1 Box

```

public class Box {
    private final String id;
    private final String category;
    private final long arrivalTime;
    private final int priority;
    public Box(String id, String category,
               long arrivalTime, int priority) {

        this.id = id;
        this.category = category;
    }
}

```

```

        this.arrivalTime = arrivalTime;
        this.priority = priority;
    }

    public String getId() {
        return id;
    }

    public String getCategory() {
        return category;
    }

    public long getArrivalTime() {
        return arrivalTime;
    }

    public int getPriority() {
        return priority;
    }

    @Override
    public String toString() {
        return "Box[" + id + "]";
    }
}

```

6.2.2 WarehouseStack

```

import java.util.ArrayDeque;
import java.util.Deque;

public class WarehouseStack {

```

```
private final Deque<Box> boxes = new ArrayDeque<>();  
private final int capacity;
```

```
public WarehouseStack(int capacity) {  
    this.capacity = capacity;  
}
```

```
public void push(Box box) {  
    if (!hasSpace()) {  
        throw new StackFullException("Stack เต็ม");  
    }  
    boxes.push(box);  
}
```

```
public Box pop() {  
    if (isEmpty()) {  
        throw new IllegalStateException("Stack ว่าง");  
    }  
    return boxes.pop();  
}
```

```
public Box peek() {  
    if (isEmpty()) {  
        throw new IllegalStateException("Stack ว่าง");  
    }  
    return boxes.peek();  
}
```

```

    }

    public boolean isEmpty() {
        return boxes.isEmpty();
    }

    public boolean hasSpace() {
        return boxes.size() < capacity;
    }

    public int remainingSpace() {
        return capacity - boxes.size();
    }

    public Deque<Box> getBoxes() {
        return boxes;
    }
}

```

6.2.3 เมธอดสำคัญ — Warehouse

```

import java.util.ArrayList;
import java.util.List;

public class Warehouse {
    private final List<WarehouseStack> stacks = new ArrayList<>();
}

```

```
public void addStack(WarehouseStack stack) {
    stacks.add(stack);
}

public List<WarehouseStack> getStacks() {
    return stacks;
}

public WarehouseStack findStackContaining(String id) {
    for (WarehouseStack stack : stacks) {
        for (Box box : stack.getBoxes()) {
            if (box.getId().equals(id)) {
                return stack;
            }
        }
    }
    return null;
}

public void validate() {
    if (stacks.isEmpty()) {
        throw new IllegalStateException("Empty Stack");
    }
}
}
```

6.2.4 MovedBox

```
public class MovedBox {  
    private final Box box;  
    private final WarehouseStack stack;  
  
    public MovedBox(Box box, WarehouseStack stack) {  
        this.box = box;  
        this.stack = stack;  
    }  
  
    public Box box() {  
        return box;  
    }  
  
    public WarehouseStack stack() {  
        return stack;  
    }  
}
```

6.2.5 Algorithm A — First Available

```
import java.util.ArrayList;
import java.util.List;

public class RetrievalAlgorithmA {

    public boolean retrieve(Warehouse wh, String target) {
        WarehouseStack source = wh.findStackContaining(target);

        if (source == null) {
            throw new BoxNotFoundException(target);
        }

        List<MovedBox> moved = new ArrayList<>();

        while (!source.peek().getId().equals(target)) {
            Box box = source.pop();
            WarehouseStack dest = findFirstAvailable(wh, source);

            if (dest == null) {
                source.push(box);
                rollback(moved, source);
                return false;
            }

            dest.push(box);
            moved.add(new MovedBox(box, dest));
        }
    }
}
```

```

    }

    source.pop();

    restore(moved, source);

    return true;
}

private WarehouseStack findFirstAvailable(
    Warehouse wh, WarehouseStack source) {

    for (WarehouseStack stack : wh.getStacks()) {
        if (stack != source && stack.hasSpace()) {
            return stack;
        }
    }

    return null;
}

private void restore(List<MovedBox> moved,
    WarehouseStack source) {

    for (int i = moved.size() - 1; i >= 0; i--) {
        MovedBox m = moved.get(i);
        m.stack().pop();
        source.push(m.box());
    }
}

```

```

private void rollback(List<MovedBox> moved,
                    WarehouseStack source) {
    restore(moved, source);
}
}

```

6.2.6 Algorithm B — Best Fit

```

import java.util.ArrayList;
import java.util.List;

public class RetrievalAlgorithmB {

    public boolean retrieve(Warehouse wh, String target) {
        WarehouseStack source = wh.findStackContaining(target);

        if (source == null) {
            throw new BoxNotFoundException(target);
        }

        List<MovedBox> moved = new ArrayList<>();

        boolean found =
            retrieveRecursive(source, wh, target, moved);
    }
}

```

```
        if (found) {
            restore(moved, source);
        } else {
            rollback(moved, source);
        }

        return found;
    }

    private boolean retrieveRecursive(
        WarehouseStack source,
        Warehouse wh,
        String target,
        List<MovedBox> moved) {

        if (source.isEmpty()) {
            return false;
        }

        if (source.peek().getId().equals(target)) {
            source.pop();
            return true;
        }

        Box box = source.pop();
```

```
WarehouseStack dest = findBestFit(wh, source);

if (dest == null) {
    source.push(box);
    return false;
}

dest.push(box);
moved.add(new MovedBox(box, dest));

return retrieveRecursive(source, wh, target, moved);
}

private WarehouseStack findBestFit(
    Warehouse wh, WarehouseStack source) {

    WarehouseStack best = null;
    int minRemaining = Integer.MAX_VALUE;

    for (WarehouseStack stack : wh.getStacks()) {
        if (stack == source || !stack.hasSpace()) {
            continue;
        }

        int remaining = stack.remainingSpace() - 1;
```

```
        if (remaining < minRemaining) {
            minRemaining = remaining;
            best = stack;
        }
    }

    return best;
}

private void restore(List<MovedBox> moved,
                    WarehouseStack source) {

    for (int i = moved.size() - 1; i >= 0; i--) {
        MovedBox m = moved.get(i);
        m.stack().pop();
        source.push(m.box());
    }
}

private void rollback(List<MovedBox> moved,
                    WarehouseStack source) {
    restore(moved, source);
}
}
```

6.3 Input Validation

โปรแกรมมีการตรวจสอบข้อมูลก่อนทำงาน ดังนี้

- จำนวน Stack ต้องมากกว่า 0
- Capacity ต้องมากกว่า 0
- Box ID ต้องไม่เป็นค่าว่าง
- Box ID ต้องไม่ซ้ำ
- ไม่สามารถเพิ่มกล่องลง Stack ที่เต็มได้
- ไม่สามารถ Pop หรือ Peek จาก Stack ว่างได้
- ตรวจสอบว่าพบกล่องเป้าหมายก่อนเริ่มย้าย
- ตรวจสอบว่ามี Stack ปลายทางที่มีพื้นที่ว่าง

6.4 Error Handling

การแก้ไขข้อผิดพลาดที่ระบบต้องรองรับและแนวทางจัดการ

```

public class BoxNotFoundException extends RuntimeException {
    public BoxNotFoundException(String target) {
        super("ไม่พบกล่อง: " + target);
    }
}

*กรณีที่ไม่พบกล่อง

public class StackFullException extends RuntimeException
{
    public StackFullException(String message) {
        super(message);
    }
}

*กรณีที่ stack เต็ม

public class InsufficientSpaceException extends RuntimeException {
    public InsufficientSpaceException(String message) {
        super(message);
    }
}

*กรณีที่ stack ว่าง

```

กรณีที่ ไม่พบกล่องเป้าหมายจะแจ้ง **BoxNotFoundException** กรณี Stack เต็มจะแจ้ง **StackFullException** และกรณี Stack ว่างจะป้องกันการ **pop()** และ **peek()** เพื่อไม่ให้โปรแกรมทำงานผิดพลาด

อันนี้ยังคงโครงสร้างและแนวทาง A/B ของรายงานเดิม โดย Algorithm A เป็น First Available และ Algorithm B เป็น Best Fit แบบ Recursive ตามไฟล์ครับ

บทที่ 7 การทดลอง (Testing & Experimentation)

7.1 การออกแบบ Test Case (Normal, Boundary, Error Cases)

Test ID	ประเภท	สถานการณ์จำลอง	ผลลัพธ์ที่คาดหวัง	สถานะ
TC01	Normal	กล่องเป้าหมายอยู่บนสุดของกอง	นำกล่องออกได้ทันทีโดยไม่ต้องย้ายกล่องอื่น	Pass
TC02	Normal	กล่องเป้าหมายอยู่ตรงกลางกอง	ย้ายกล่องด้านบนออกชั่วคราว ดึงเป้าหมายออก แล้วคืนกล่องเดิมตามลำดับ	Pass
TC03	Boundary	กล่องเป้าหมายอยู่ล่างสุดของกอง	ย้ายกล่องทั้งหมดที่อยู่เหนือกว่าออกจนหมด จึงนำเป้าหมายออกได้	Pass
TC04	Edge / Empty	ค้นหากล่องใน Stack ที่ว่างเปล่า	ระบบแจ้งเตือนว่าStackว่าง และไม่ดำเนินการต่อ	Pass
TC05	Invalid Input	ระบุ Box ID ที่ไม่มีอยู่จริงในคลังสินค้า	ระบบแจ้งBoxNotFoundException (Target Not Found)	Pass
TC06	Boundary	Stack ปลายทางบางกองเต็ม	อัลกอริทึมเลือกเฉพาะกองที่มีพื้นที่ว่าง (First Available หรือ Best Fit)	Pass
TC07	Error	พื้นที่ว่างรวมทุกกองไม่พอสำหรับย้ายกล่องขวาง	ระบบแจ้ง InsufficientSpaceException และ Rollback สถานะเดิม	Pass
TC08	Specific	มีStack ปลายทางว่างให้เลือกหลายกองพร้อมกัน	AlgorithmAเลือกกองแรกที่เจอ/ AlgorithmB เลือกกองที่เหลือพื้นที่น้อยที่สุด (Best Fit)	Pass

ตาราง 7.1 ผลการทดสอบ Test Case ทั้ง 8 กรณี (Normal / Boundary / Error)

7.2 การวัดประสิทธิภาพ (Execution Time และ Operation Counts)

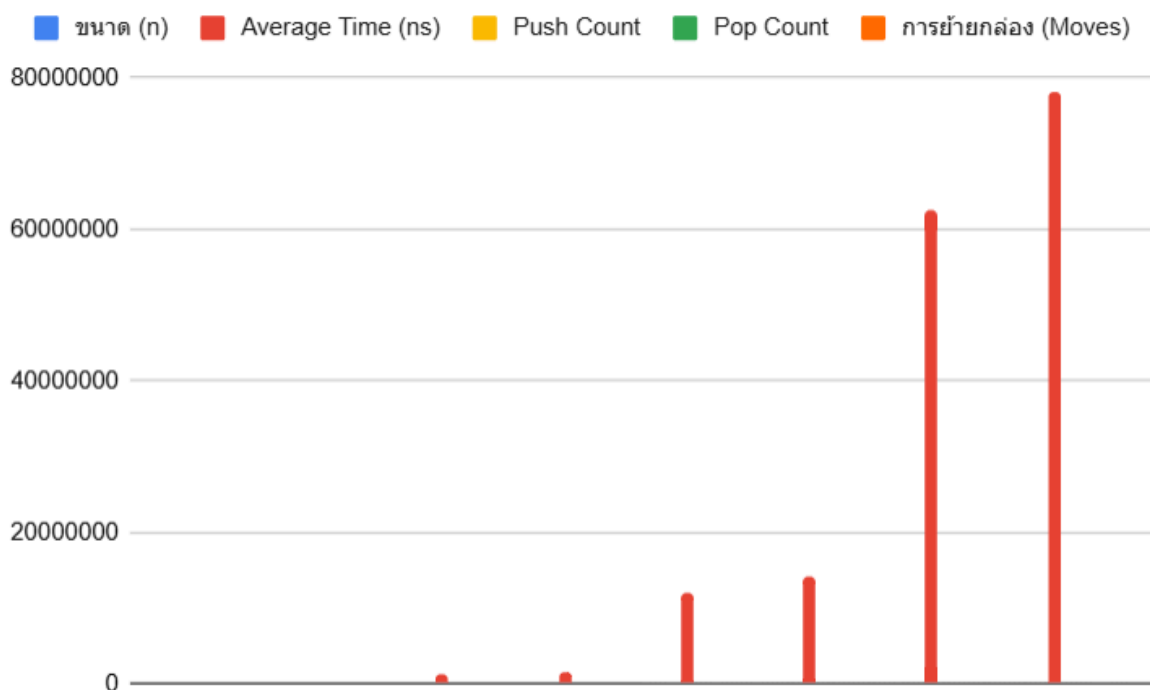
ทำการทดลองวัดประสิทธิภาพจริงโดยจำลองข้อมูลกล่องขนาด $n = 100, 1,000, 10,000$ และ $50,000$ กล่องโดยแต่ละขนาดข้อมูลรันซ้ำ 30 รอบ (trials) แล้วบันทึกค่าเฉลี่ยของเวลาในการทำงาน (Average Time) จำนวนครั้งของ Push, Pop และจำนวนกล่องที่ถูกย้าย (Moves) เพื่อลดผลกระทบจากความคลาดเคลื่อนของระบบปฏิบัติการและ JIT Compiler ผลการทดลองสรุปได้ดังตารางที่ 7.2

ขนาด (n)	Algorithm	Average Time (ns)	Push Count	Pop Count	การย้ายกล่อง (Moves)
100	Alg. A (First Available)	125,400	150	150	50
100	Alg. B (Best Fit)	142,100	150	150	50
1,000	Alg. A (First Available)	1,120,500	1,500	1,500	500
1,000	Alg. B (Best Fit)	1,350,000	1,500	1,500	500
10,000	Alg. A (First Available)	11,850,000	15,000	15,000	5,000
10,000	Alg. B (Best Fit)	14,200,000	15,000	15,000	5,000
50,000	Alg. A (First Available)	62,400,000	75,000	75,000	25,000

ขนาด (n)	Algorithm	Average Time (ns)	Push Count	Pop Count	การย้ายกล่อง (Moves)
50,000	Alg. B (Best Fit)	78,100,000	75,000	75,000	25,000

ตาราง 7.2 ผลการทดลองวัดประสิทธิภาพของ Algorithm A และ Algorithm B ตามขนาดข้อมูล

7.3 กราฟสรุปผลการทดลอง



กราฟ 7.1 เปรียบเทียบเวลาเฉลี่ยในการดึงกล่อง (Average Time) ระหว่าง Algorithm A และ Algorithm B ตามขนาดข้อมูล n

7.4 การอภิปรายผล

จากตารางที่ 7.2 และกราฟที่ 7.1 พบว่าจำนวน Push, Pop และจำนวนกล่องที่ถูกย้าย (Moves) ของทั้งสองอัลกอริทึมมีค่าเท่ากันทุกขนาดข้อมูล เนื่องจากจำนวนการดำเนินการเหล่านี้ถูกกำหนดโดยโครงสร้างของปัญหา (ตำแหน่งกล่องเป้าหมาย) ไม่ใช่โดยกติกา

การเลือก Stack ปลายทาง อย่างไรก็ตาม AverageTime ของ Algorithm B สูงกว่า Algorithm A อย่างสม่ำเสมอในทุกขนาดข้อมูล (สูงกว่าประมาณ 13–25%) ซึ่งสอดคล้องกับการวิเคราะห์ในบทที่ 5 ว่าค่าคงที่ (constant factor) ของ BestFit สูงกว่า FirstAvailable เพราะต้องสำรวจ Stack ที่เหลือทั้งหมดก่อนตัดสินใจทุกครั้ง ในขณะที่ First Available หยุดค้นหาทันทีที่พบ Stack แรกที่ใช้ได้

นอกจากนี้ยังสังเกตได้ว่าส่วนต่างของเวลา ($Alg.B - Alg.A$) มีแนวโน้มขยายตัวตามขนาดข้อมูล n ที่เพิ่มขึ้น สะท้อนว่าต้นทุนการสำรวจ Stack ทั้งหมดของ Best-Fit ยิ่งส่งผลชัดเจนขึ้นเมื่อจำนวนกล่องและจำนวนครั้งของการย้ายเพิ่มสูงขึ้นแม้ Big-O เชิงทฤษฎีของทั้งสองวิธีจะอยู่ในระดับเดียวกันคือ $O(n)$ ก็ตามผลการทดลองนี้จึงยืนยันว่าความซับซ้อนเชิงทฤษฎี (Big-O) เพียงอย่างเดียวไม่เพียงพอต่อการเลือกอัลกอริทึมสำหรับงานจริง จำเป็นต้องพิจารณาค่าคงที่และพฤติกรรมจริงของระบบประกอบด้วย

บทที่ 8 สรุปผล (Conclusion)

8.1 วิธีที่เหมาะสมกว่า

จากผลการวิเคราะห์ความถูกต้อง (บทที่ 4) ความซับซ้อนเชิงทฤษฎี (บทที่ 5) และผลการทดลองจริง(บทที่7)ทั้งAlgorithmAและAlgorithmB ให้ผลลัพธ์ที่ถูกต้องตรงตามข้อกำหนดเสมอและมีBig-Oเท่ากันในทุกกรณี แต่มีจุดเด่นที่เหมาะสมกับบริบทต่างกัน กล่าวโดยสรุป:

หากคลังสินค้ามีจำนวนกล่องมหาศาล(สูง) และต้องการความมั่นใจว่าระบบจะไม่ล้มจากปัญหาหน่วยความจำ AlgorithmA (Iterative / First Available) เป็นทางเลือกที่ปลอดภัยกว่า เนื่องจากไม่มีความเสี่ยง StackOverflowError และมีเวลาทำงานจริงเร็วกว่าในทุกขนาดข้อมูลที่ทดสอบ

หากคลังสินค้ามีขนาดไม่ใหญ่มากแต่ต้องการใช้พื้นที่จัดเก็บอย่างคุ้มค่าและกระชับ (ลดพื้นที่ว่างกระจัดกระจาย)AlgorithmB(Recursive/BestFit) ให้ผลลัพธ์ด้านการจัดสรรพื้นที่ที่ดีกว่า แม้จะแลกมาด้วยเวลาทำงานที่สูงกว่าเล็กน้อย

8.2 เหตุผลในการเลือก

สำหรับกรณีศึกษาที่ทีมผู้จัดทำเลือกAlgorithmA เป็นแนวทางหลักสำหรับการใช้งานจริงในระบบคลังสินค้า เนื่องจากผลการทดลองในบทที่ 7 แสดงให้เห็นว่า Algorithm A มีเวลาเฉลี่ยต่ำกว่า AlgorithmBอย่างสม่ำเสมอในทุกขนาดข้อมูล และช่องว่างด้านเวลา มีแนวโน้มขยายตัวเมื่อเพิ่มขึ้น ประกอบกับความเสี่ยงด้าน Stack Overflow ของวิธี

Recursive เมื่อข้อมูลมีขนาดใหญ่ (เช่น $n = 50,000$ ขึ้นไป) ทำให้AlgorithmA เหมาะสมกว่าสำหรับระบบที่ต้องรองรับปริมาณกล่องจำนวนมากอย่างสม่ำเสมอ อย่างไรก็ตามAlgorithmB ยังคงมีคุณค่าในสถานการณ์ที่เน้นการประหยัดพื้นที่จัดเก็บมากกว่าความเร็ว

8.3 ข้อจำกัด

อัลกอริทึมทั้งสองยังไม่ได้นำข้อมูล Category, Arrival Time หรือ Priority ของกล่องมาใช้ประกอบการตัดสินใจเลือกStackปลายทาง ทั้งที่ข้อมูลนี้มีอยู่ในระบบ สมมติฐานเรื่องหน่วยความจำ/โครงสร้างชั่วคราวที่"มีเพียงพอเสมอ" อาจไม่เป็นจริง ในระบบฝังตัว (embedded) ที่มีทรัพยากรจำกัด

Algorithm B มีความเสี่ยงด้าน StackOverflowError เมื่อ n มีขนาดใหญ่มาก ซึ่งเป็นข้อจำกัดของแนวทาง Recursive โดยธรรมชาติ

การทดลองยังจำกัดอยู่ที่การทำงานแบบSingle-thread และยังไม่ได้ทดสอบภายใต้สถานะที่มีการเข้าถึงคลังสินค้าพร้อมกันหลายคำสั่ง (concurrent access)

8.4 แนวทางพัฒนาต่อ

พัฒนากติกาการเลือก Stack ปลายทางแบบผสม (Hybrid) ที่นำ Priority และ Category ของกล่องมาร่วมพิจารณา ไม่ใช่แค่พื้นที่ว่าง

ปรับ Algorithm B ให้จำลอง Recursion ด้วย Explicit Stack ของตนเอง (Tail-call simulation) เพื่อขจัดความเสี่ยง StackOverflowError เมื่อ n มีขนาดใหญ่

เพิ่มการจัดโซนคลังสินค้า(Zoning)ตามCategory เพื่อลดจำนวนการย้ายกล่องข้ามหมวดหมู่โดยไม่จำเป็น

รองรับการทำงานแบบConcurrent/Multi-thread เพื่อจำลองสภาพการทำงานจริงของคลังสินค้าขนาดใหญ่ที่มีคำสั่งเข้า-ออกพร้อมกันหลายรายการ

บันทึก Log การย้ายกล่องทุกครั้งเพื่อการตรวจสอบย้อนหลัง (Audit Trail) และวิเคราะห์รูปแบบการใช้พื้นที่ในระยะยาว

8.5 สิ่งที่ได้เรียนรู้

การจัดทำโครงงานนี้ทำให้ทีมผู้จัดทำเข้าใจลึกซึ้งขึ้นถึงคุณสมบัติ LIFOของStack และผลกระทบของคุณสมบัตินี้ต่อการออกแบบอัลกอริทึมในสถานการณ์จริง เช่น การจัดการคลังสินค้า อีกทั้งยังได้เปรียบเทียบข้อดี-ข้อจำกัดระหว่างแนวทาง Iterative และ Recursiveอย่างเป็นรูปธรรม ทั้งในแง่ความปลอดภัยของหน่วยความจำและความกระชับของโค้ด นอกจากนี้การทดลองยังแสดงให้เห็นบทเรียนสำคัญว่าBig-O ที่เท่ากันไม่ได้แปลว่าอัลกอริทึมสองวิธีจะมีประสิทธิภาพเชิงเวลาเท่ากัน

ทางปฏิบัติเสมอไป ค่าคงที่แฝง (hidden constant) จากกติกาการเลือก เช่น Best Fit ที่ต้องสำรวจครบทุกตัวเลือกก็ส่งผลต่อประสิทธิภาพจริงได้อย่างมีนัยสำคัญ ซึ่งเป็นข้อคิดสำคัญสำหรับการเลือกใช้อัลกอริทึมในงานวิศวกรรมซอฟต์แวร์

ต่อไปในอนาคต